Technical application architecture of Pseudalis Community

A local Windows service that finds personal data with four detection devices and replaces it by format-preserving encryption

Aldric Pierrain
Cydral Technology, Paris, France
cydraltechnology@gmail.com

8 November 2026

Abstract

Pseudalis Community protects the personal data of documents on the Windows computer that holds them and is distributed under the Boost Software License 1.0. A LocalSystem service hosts an engine library, a mapped drive, an encrypted vault with a SQLite store, a key manager, a signed updater and an HTTPS interface on the loopback address, which Explorer commands, a tray icon and a web application call. The engine finds personal data with four detection devices: validated scanners with field labels, user patterns and imposed terms; 44 internal tables in five languages; a structural SVM sequence tagger; and a byte-level transformer encoder of 116 791 parameters. A pseudonymizer added to dlib replaces each value by a tag that carries its FF1 encryption under AES-256 (NIST SP 800-38G) and a 15-bit check, or by the bare substitute, with codecs that keep check digits valid, shift dates by a secret number of weeks and encrypt IP addresses with Crypto-PAn. On a synthetic test set of 5000 examples in five languages, the whole detector with the current models reaches F1 0.945, against 0.721 for the first models, trained on the French and English corpus, and reads about 41 000 characters per second on two cores. On 160 hand-written texts, F1 ranges from 0.889 to 0.899. The output remains pseudonymized personal data for the holder of the key, which the report relates to the GDPR, to EDPB guidance as of November 2026 and to the case law of the Court of Justice.

Keywords: *pseudonymization; personal data detection; format-preserving encryption; FF1; structural SVM; byte-level transformer; GDPR; dlib*

1. Introduction

Pseudalis Community protects the personal data of documents on a Windows computer, on that computer alone [1]. Its version header names the author, Aldric Pierrain, and the notice "Copyright (C) 2026 Cydral Technology" (`common/version.h`). The program is distributed under the Boost Software License 1.0, and its installer and signed update feed are published in the `dist/pseudalis` folder of the repository [2]. The README describes the protection as a pseudonymization in the sense of the GDPR [1].

Pseudalis rests on the Dlib-Transformer-extensions repository [3], a fork of the dlib C++ library [4]. Its engine compiles the module `dlib/privacy` [5], which holds the detector, two learned recognizers, a format-preserving pseudonymizer and the handlers of Office, PDF and text documents. The example program `examples/slm_document_anonymizer_ex.cpp` exposes the same module on the command line and trains and evaluates the recognizers. The pseudonymizer implements FF1 of NIST SP 800-38G [6] against the constraints of its second revision draft [7]. A coverage matrix maps the detected categories to the GDPR [8, 9], and evaluation reports give the scores of the successive models [10].

The description rests on the source code read on 8 November 2026 and cited by path, on runs of the example program on synthetic sentences and of a demonstration program on synthetic values under a synthetic key, on the evaluation reports, and on time measurements taken that day on a virtual machine with two cores. Statements derived from the code without a run are marked as inferences. Paths such as `host/service.cpp` are relative to `dlib/dlib/external/pseudalis/`, and library paths follow the include convention of dlib, such as `dlib/privacy/pii_detector.h`. Section 2 describes the application, Section 3 detection, Section 4 the pseudonymization, Section 5 the dataset and Section 6 the performance. Section 7 relates the output to the GDPR as a technical description that is not legal advice.

2. Application architecture

2.1 Programs, engine libraries and their loading

Pseudalis installs one Windows service, three programs of the user session and a web application (Table 1). The service `pseudalis-service.exe` runs as LocalSystem and holds every part that keeps state [1]. Figure 1 shows these components and the two entry paths of documents, the HTTPS interface and the drive. Two parts lie outside the service, the signed update feed (Section 2.5) and Albireo, an optional local inference service built on the same library, which can describe pictures (Section 2.6).

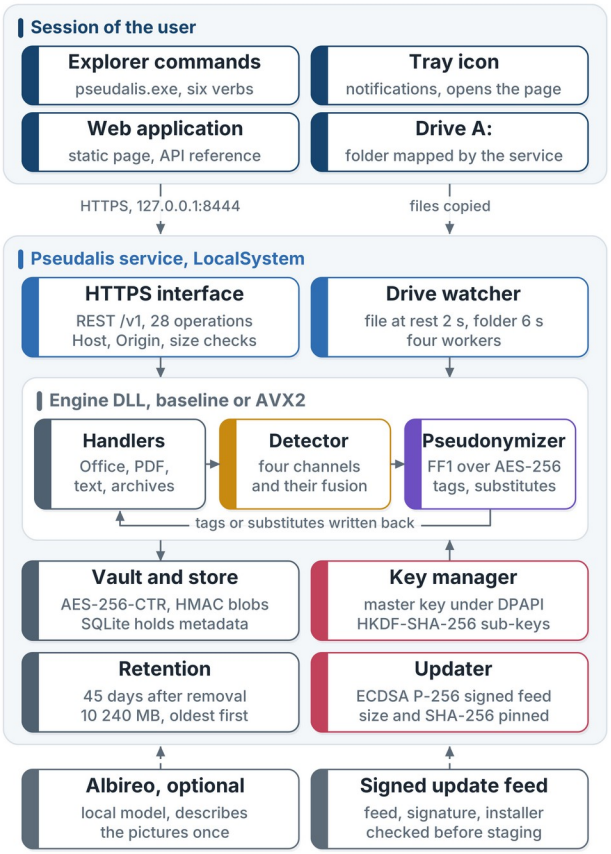


Figure 1. Components of Pseudalis Community: programs of the user session, the LocalSystem service with its engine library, and the optional Albireo service and signed update feed outside it.

File	Role
`pseudalis-service.exe`	LocalSystem service: engine host, drive, vault, database, HTTPS interface, updater
`pseudalis-engine.dll`	engine for every x64 processor
`pseudalis-engine-avx2.dll`	engine compiled for AVX2 and FMA
`pseudalis-tray.exe`	tray icon, started at each sign-in
`pseudalis.exe`	Explorer commands and command line
`pseudalis-setup.exe`	installer, kept as uninstaller
`web\`	web application and interface documentation

Table 1. Files installed by Pseudalis Community and their roles, from the README [1].

The two engine libraries are built from the same sources, one for every x64 processor and one compiled for AVX2 and FMA (`CMakeLists.txt`). Both export the same C interface of eleven functions. `psd_create` takes the 32-byte pseudonymization key, and `psd_process` runs operations such as `anonymize`, `restore`, `detect` and `extract` (`common/engine_api.h`). The trained models are compiled into both libraries from `examples/slm_document_anonymizer_ex.h`.

The service selects the AVX2 library when CPUID and `XGETBV` report AVX2, FMA and operating-system support (`common/engine_loader.cpp`). Before any code of a library runs, the loader maps it as a data file and refuses a build stamp that differs from its own, a stamp that each CMake configuration draws from the UTC time and eight random hexadecimal digits. It then loads the library by full path from the program folder and System32 and checks the interface level and the build string. By inference from this order, a planted or stale library is refused before its `DllMain` runs, and a mismatching AVX2 library makes the start fail with no fall back. The programs link their runtime statically and load their dependencies from System32, and the tool `pseudalis-check-imports` stops a build whose programs import anything other than Windows system libraries (`tools/check_imports.cpp`).

2.2 Local HTTPS service and REST interface

The service runs a cpp-httplib server over Mbed TLS on 127.0.0.1, port 8444 by default (`host/service.cpp`). Its certificate comes from a local EC P-256 authority created for each installation and limited by name constraints to localhost and the loopback addresses, whose private key signs once and is erased without ever being written (`host/certificates.cpp`). Before reading any body, a filter answers 421 to a foreign `Host`, 403 to a foreign `Origin` and 413 to an oversized `Content-Length` (`host/server.cpp`). Responses carry `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` and a content security policy limited to the page's own scripts. These checks stop web pages in the user's browser from driving the service, and they do not stop a local process, which the README accepts as the trust model of the computer [1].

The interface exposes 28 operations under `/v1`, described by an OpenAPI 3.0 document with a Swagger UI page (`web/openapi.json`). Table 2 lists the main routes. File, text and name operations need no token. Administration routes require the header `X-Pseudalis-Admin` with 48 hexadecimal digits drawn from 24 random bytes and kept in `admin.token`, which interactive users can read (`common/platform.cpp`). Work slots number min(8, cores/2), and a request that waits 60 s for a slot receives 503 with `Retry-After: 15`. The answer is the protected file with a JSON report in the header `X-Pseudalis-Report`, or JSON with base64 data.

Route	Token	Use
`POST /v1/anonymize`	no	protect a file
`POST /v1/restore`	no	restore a file
`GET /v1/lookup`	no	find a record by hash
`POST /v1/text/anonymize`	no	protect a text
`POST /v1/names/anonymize`	no	protect a file name
`GET /v1/info`	no	status, key fingerprint
`GET /v1/records`	yes	list the records
`DELETE /v1/records/{id}`	yes	forget a record
`PUT /v1/settings`	yes	change the settings
`PUT /v1/terms`	yes	set the imposed terms
`POST /v1/key/export`	yes	export the key

Route	Token	Use
`POST /v1/updates/apply`	yes	install a staged update

Table 2. Main routes of the interface, from `host/server.cpp` and `web/openapi.json` [1].

2.3 Drive A:, Explorer commands, tray icon and web application

The drive involves no file-system driver, which would need Microsoft signing for each version [1]. The service maps the folder `<data>\drive` to a letter, A: by default, with `DefineDosDeviceW`, so the letter appears in every session (`host/drive_folders.cpp`). A watcher combines `ReadDirectoryChangesW` with periodic rescans, and up to min(4, cores/2) workers process a file once it has rested 2 s and can be opened alone. The service refuses files above `max_file_mb`, 150 MB by default, and files of unknown kind that are not text. It keeps as they are the exact copies of protected files, recognized by SHA-256, and adopts the entries of recorded archives. It protects any other file with a record, writes the output under a hidden temporary name, renames it and deletes the written file through its handle. Refused files move to `<data>\refused\<SID>`, readable only by their owner, SYSTEM and administrators. Without a driver, the written file lies in clear on the disk until it is processed, and its deleted sectors are freed without being overwritten [1].

The installer registers a cascade menu whose six verbs run `pseudalis.exe` to protect, protect without tags, protect a copy with or without tags, restore and restore a copy (`windows/setup_actions.cpp`). Two session mutexes let at most two such processes work at a time. After writing a protected file outside the drive, the program records its NTFS object identifier, which later identifies an edited copy (`windows/shell.cpp`). The tray icon polls the service, shows notifications and opens the web application with the administration token in the fragment of the address, which the page moves to `sessionStorage` (`web/js/app.js`). The page protects files, folders packed as ZIP archives and texts, restores them, and offers administration cards for records, key, settings and imposed terms.

The engine accepts Office Open XML documents in 14 extensions, PDF, 28 text extensions, ten picture extensions, ZIP, TAR and TAR.GZ archives, and text of any extension that decodes cleanly and holds no encoded part such as base64 or PEM (`common/file_kinds.h`). An archive or Office file may expand to at most 64 times its size, within 16 MB and 2 GB, and an archive holds at most 20 000 entries (`engine/archives.h`).

2.4 Vault, SQLite store and key management

The vault keeps originals, pictures and picture descriptions as blobs that hold the marker `PSDV1`, a random 16-byte IV, the AES-256-CTR ciphertext and an HMAC-SHA-256 over the blob identifier and every preceding byte [11, 12, 13]. The service verifies the MAC in constant time before decryption (`host/vault.cpp`). The SQLite database runs with `journal_mode=WAL`, `synchronous=FULL` and secure delete. It holds metadata only: names, sizes, hashes, places, links to pictures and archive entries, events, imposed terms and the pairs of substitute and tag, never a content or a value in clear (`host/store.cpp`).

At its first start, the service draws a 32-byte master key and stores it as a DPAPI blob of the LocalSystem account in a folder restricted to SYSTEM and administrators (`host/keys.cpp`). HKDF-SHA-256 [14] derives from it the pseudonymization key and the two vault keys, and the engine derives a picture key, so picture identifiers do not link pictures across installations (`engine/engine.cpp`). DPAPI binds the blob to one computer, so by inference a data folder moved to another computer without an export loses its key. The export seals the key under a passphrase of at least 12 characters with PBKDF2-HMAC-SHA-256 over 600 000 iterations, the count that OWASP recommends [15], followed by AES-256-CTR and HMAC-SHA-256. Import works only on an installation without records. The README calls the export the backup of the installation and states the trust model. Every user of the computer shares its key, its originals and its administration, so people who must not see each other's documents need separate computers [1].

2.5 Signed updates

The updater reads `feed.json` and its signature from the `dist/pseudalis` folder of the repository [2], 5 minutes after the start and then every 24 h (`host/updater.cpp`). It verifies an ECDSA P-256 signature over the exact bytes of the feed against a public key compiled into the service. It downloads the installer that the feed names, checks its size and SHA-256, stages it with the signed feed, checks both again right before the launch and runs it with `/update /silent`. The script `tools/publish_release.py` signs the feed with a private key kept outside the repository. A search of `host/updater.cpp` finds no Authenticode check, so the integrity of an update rests on the feed signature, which pins the size and hash of the installer.

2.6 Protection, restoration, retention and forgetting

Protection detects the personal data of a document, writes the protected file and, when the original is kept, creates a record that stores the original in the vault and its names, sizes and hashes in the database (`protect()` in `host/service.cpp`). Figure 2 shows this path and the order of restoration. Values in the content become tags `[[TYPE:body#CCC]]`, while file names, archive entry names and document properties take substitutes without brackets, recorded with their tags (`engine/engine.h`).

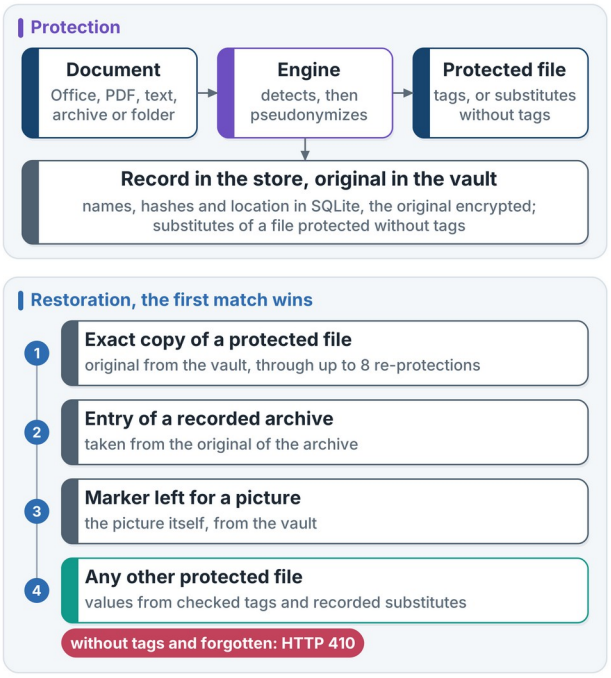


Figure 2. Lifecycle of a document: protection with its record, and the four restoration paths tried in order, with HTTP 410 for a file without tags whose record was forgotten.

Pictures leave the documents. The engine takes each picture out of a Word run, a PowerPoint text box or a PDF page, stores it in the encrypted vault and writes the marker `[Image@<id>]` in its place, or a gray box in a PDF (`engine/engine.h`). Pseudalis analyzes no picture content and performs no OCR, so a scanned document loses its text [1]. Albireo, another product of the author, is a local inference service built on the same Dlib-Transformer-extensions library and running on the same computer. When it is installed and one of its models reads images, Pseudalis asks it once for a description of each picture, pseudonymizes the description like any text and keeps it encrypted in the vault, so the protected document carries `[Image@<id> : <description>]` in place of the picture while the picture itself stays in the vault.

Restoration tries four paths, and the first match wins (`service::restore()`). An exact copy of a protected output, found by its SHA-256, returns the original from the vault, through chains of up to 8 successive protections. An entry of a recorded archive returns the original entry, and a marker file returns its picture. Any other file goes through the engine, which restores the tags whose check verifies and the substitutes that the database knows. The server answers 404 for a file without protected values, 410 for a forgotten picture or a forgotten file without tags, and 422 for other failures (`host/server.cpp`). Removed pictures, PDF fonts and embedded objects never come back from tags [9].

Retention combines a time limit and a space budget (`host/settings.h`). The time limit, 45 days by default, starts when the last protected copy of a record leaves the drive. The space budget, 10 240 MB by default, forgets the oldest records first, 64 at a time, and keeps 256 MB of free disk space when forgetting can provide it. By inference from the ordering clause, a record whose file is still on the drive can go before a newer record whose file was deleted. Forgetting, by a user or by these limits, runs in one transaction that deletes the record, its entries, its substitutes and its unshared pictures, and leaves a `forgotten` row with the output hash (`host/store.cpp`). A later copy of a forgotten file with tags is restored from its tags only, without pictures, and a forgotten file without tags receives HTTP 410 with the date of forgetting.

2.7 Protection without tags

The current release adds protection without tags [9]. The setting `tags`, true by default, governs the drive and the requests that do not state it, and the web cards and the Explorer verbs with `--no-tags` override it. Without tags, the engine writes each value's substitute alone, the body of its tag without brackets, keyword or check (`dlib/privacy/document_anonymizer_abstract.h`), so nothing in the file tells a value from its substitute. The record keeps the encrypted original and the pairs of substitute and tag, never a value in clear.

Restoration then depends on the record. An exact copy returns the original. An edited copy is restored when the service finds its record through the NTFS object identifier, after which the engine looks up every run of one to twelve words among the recorded substitutes and accepts a tag only when its check verifies (`find_substitutes()` in `dlib/privacy/pseudonymizer.h`). A file protected without tags and without a record reports `"restorable": false`, and a forgotten one receives 410 `tagless_forgotten`. By design, a recorded substitute that reappears in an edited copy is restored there too, for example a shifted date typed again, a limit that the coverage matrix states [9].

3. Detection of personal data

3.1 Entity types and the two readings of a document

The detector reports 21 types, each with a tag keyword: `PERSON`, `ORG`, `LOCATION`, `ADDRESS`, `EMAIL`, `PHONE`, `IBAN`, `CARD`, `NIR`, `SIRET`, `ID`, `IPV4`, `IPV6`, `MAC`, `URL`, `DATE`, `PLATE`, `GEO`, `TITLE`, `GENDER` and `NATIONALITY` (`dlib/privacy/pii_types.h`). The two learned recognizers know seven fine labels, `GIVEN_NAME`, `FAMILY_NAME`, `USERNAME`, `STREET_ADDRESS`, `POSTAL_CODE`, `CITY` and `ORGANIZATION`, and every structured type comes from rules, labels, patterns or imposed terms. The rules read French, English, Italian, Spanish and German.

Detection reads each document twice [5]. `learn()` first reads every unit, and `detect()` then reports the values of each unit, with the channels in a fixed order: existing tags are masked, then come scanners and dates, labels and tables, linguistic rules, user terms and patterns, and the learned recognizers (`analyze()` in `dlib/privacy/pii_detector.h`). Figure 3 shows the four detection devices and the stages that follow them, the

fusion by priority, the document propagation, the filters and the imposed terms applied over the whole text at the end.

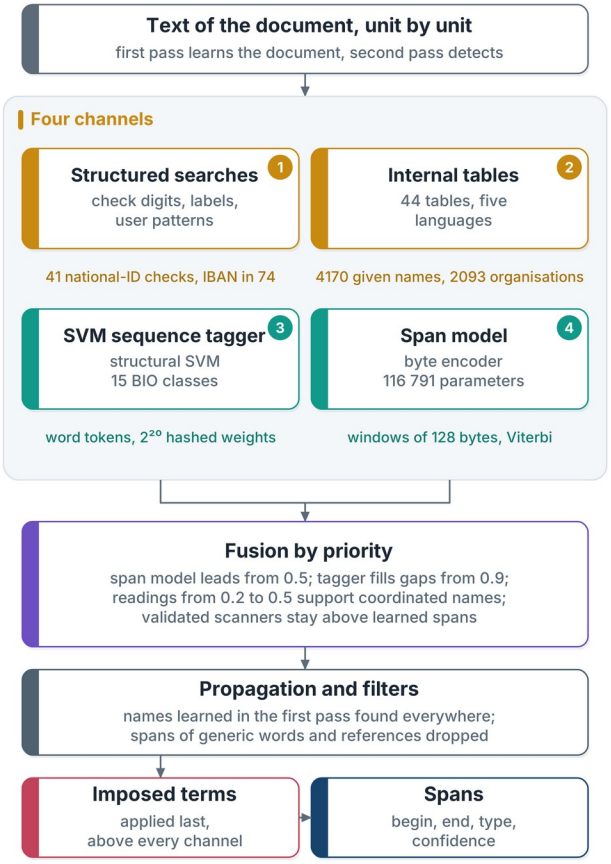


Figure 3. Detection pipeline: two readings of the document, the four detection devices, fusion by priority, document propagation and filters, and the imposed terms applied last, which yield typed spans.

3.2 Device 1: structured searches and substitutions

The validated scanners are hand-written and run in linear time (`dlib/privacy/pii_detector_abstract.h`). Each accepts a value only when its check digits verify or, for numbers with a weak check or none, after a whole-word cue. A French NIR needs a valid month field and the key 97 minus its first 13 digits modulo 97. A SIRET passes Luhn and follows a cue such as `siret` within 48 characters, a card passes Luhn with the lengths of its issuer, and an IBAN matches one of 74 country lengths and passes mod 97. `dlib/privacy/id_checks.h` adds 41 check functions for the national identifiers of 40 countries, among them Verhoeff, ISO 7064 and the eleven test of the Dutch BSN, and five VAT checks. Distinctive layouts such as the codice fiscale need no cue, numbers whose check leaves one chance in ten need one, and identity numbers without a check are found only behind their label. Dates come from `find_dates()` in five languages, which ignores dates that name streets and software versions such as `6.1.24`.

Field labels type the value that follows them. `field_labels()` maps 838 folded labels to types, and a labeled value is marked `kept`, so the filter of generic words never drops it. A handler can also force a type, for the authors of the metadata or a cell under a column headed `Nom`, with a priority bonus of 200. Organizations add regular expressions, each a `pii_pattern` with an optional `luhn` or `iban` check. Patterns run in windows of 2048 bytes that overlap by 512, which bounds the backtracking of `std::regex`, and receive a bonus of 150, above every scanner (`scan_patterns()`). User terms receive a bonus of 10.

Imposed terms come from the settings of Pseudalis, which hold up to 100 terms of at most 200 characters, typed `ORG`, `PERSON`, `LOCATION`, `ADDRESS`, `ID`, `EMAIL`, `PHONE` or `URL` and optionally case-sensitive (`common/imposed_terms.h`). The detector applies them in a final pass over the whole text, as whole runs of letters and digits, and their occurrences outrank every other channel (`impose_terms()`). The README describes the same precedence as terms replaced before anything else is read [1].

3.3 Device 2: internal tables

The header `dlib/privacy/pii_lexicons.h` holds 44 tables, 43 sets and one label map, in folded form for the five languages (Table 3). Its comment on given names states that "a first name raises the score of a candidate span, which still needs a capitalized shape and is discarded when it collides with a common word, a role or a heading". The rules turn the tables into scores. In `scan_persons()`, a known given name adds 2, a known full name 4 and a civility before the candidate 3.5, and a score of at least 2.5 yields a person with confidence min(1, score/5). Laws and standards in capitals, such as `RGPD`, `NIS2` or `ISO 27001`, are never a person, a place or an organization. The tagger reads 12 of the tables as features.

Table	Entries	Role
`common_words()`	10 428	words that never name alone, 10 880 with the business words
`first_names()`	4170	given names of all regions
`ambiguous_first_names()`	268	given names that are also common words
`known_organizations()`	2093	organizations known by name
`field_labels()`	838	label to type map
`nationality_words()`	756	nationalities
`business_words()`	572	vocabulary of tenders and reports
`country_names()`	315	countries, left readable
`public_body_acronyms()`	304	acronyms of public bodies
`product_names()`	294	products, left readable
`acronym_words()`	293	technical acronyms
`public_body_names()`	244	names of public bodies

Table 3. Main internal tables of `dlib/privacy/pii_lexicons.h`, with their sizes as counted in the header [5].

3.4 Device 3: the structural SVM sequence tagger

The class `pii_sequence_tagger` uses the chain model of dlib's `sequence_labeler` [4], trained as a structural SVM [16]

with a cutting-plane solver (`dlib/privacy/pii_sequence_tagger_abstract.h`). It labels word and punctuation tokens with 15 classes in the BIO scheme [17], the class O and a B and an I class for each fine label, and it refuses an I class that does not follow its B class or that opens a line or a cell. The features of a token cover its folded form, shapes, length, prefixes and suffixes, the table flags of the token and its neighbors, the words and shapes two positions around it, its place in the line and the type announced by a preceding label (`make_sequence()`). Hashing with FNV-1a into 2^{20} emission weights, plus 240 transition and start weights, gives 1 048 816 weights whatever the vocabulary, stored in single precision in 6 191 910 bytes. The current tagger was trained afresh, for the first time since the release that introduced the five-language corpus, with C = 10 on 80 000 examples drawn from 1 207 499, in 3763 s on two cores [10]. The confidence of a span is the logistic of its emission margin over the class O, averaged over its tokens, so by inference it ignores the transition weights.

3.5 Device 4: the byte-level span model

The class `pii_span_model<D, L, H>`, the small language model of spans of the project, is a bidirectional transformer encoder [18] over UTF-8 bytes, with a vocabulary of 260 entries and no tokenizer (`dlib/privacy/pii_span_model_abstract.h`). Each block applies RMSNorm [19] before a fused grouped-query attention [20] with rotary positions [21], and RMSNorm before a SwiGLU feed-forward layer [22]. A linear head gives each byte one of the 15 BIO classes, trained with a cross-entropy loss per token. The causal mask is off, since "whether `Rose` is a person depends on the words that follow as much as on those before" (`examples/Readme.md`). Figure 4 shows the tiny size that the engine embeds, with D = 64, two blocks, two heads of 32 dimensions and windows of 128 bytes.

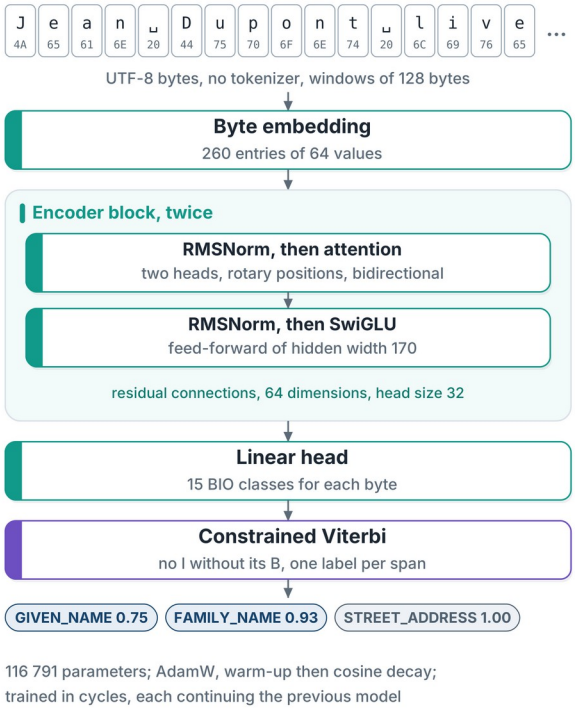


Figure 4. The span model of the engine: byte embedding of 260 entries of 64 values, two encoder blocks, a linear head of 15 BIO classes and a constrained Viterbi decoder, with the outputs of the current model on the synthetic sentence "Jean Dupont lives at 12 rue de la Paix".

The example program prints 116 791 parameters for this size, of which 16 640 form the byte embedding of 260 rows of 64 values (`parameter_count()`). By inference, the rest fits two blocks of 49 556 values, each made of two RMSNorm gains, a fused query, key and value projection of 12 288, an output projection of 4096 and a SwiGLU layer of hidden width 170 with biases, and a head of 1039 values. The width 170 is the integer part of 8 × 64 / 3, which matches the template arguments 8 and 3. The type passes the number of heads twice, which suggests as many key and value heads as query heads, so the grouped-query layer would reduce to multi-head attention at this size. The files that would confirm both readings lie outside those read for this report.

A constrained Viterbi pass turns per-byte probabilities into spans (`viterbi_bio()`). An I class may only follow its own B or I class, and line feeds and tabulations can only take the class O. The confidence of a span is the mean probability of its class over its bytes, and its ends move to whole words. Long texts are read in windows of 128 bytes with a stride of 96, each byte taking the prediction of the window where it lies most central, under `npu::exact_scope`, so the results do not depend on the hardware. On the synthetic sentence of Figure 4, the current model reads `Jean` as `GIVEN_NAME` with 0.75, `Dupont` as `FAMILY_NAME` with 0.93 and the street address with 1.00.

Training uses AdamW [23] with moments 0.9 and 0.98, a weight decay of 0.01, a linear warm-up and a cosine decay, on windows of 32, 64 and 128 bytes. The current model continues the previous one for two cycles, 30 000 steps at a peak rate of

0.0003 and then 24 000 at 0.0001, on 1 207 499 examples in 71 minutes on two shared cores, and its development loss went from 0.1024 to 0.1015 [10]. The command `embed` compresses both model files with zlib and writes them in base64 into `examples/slm_document_anonymizer_ex.h`, which decodes byte for byte to the model files and which Pseudalis compiles into its engine [9].

3.6 Fusion, document propagation and filters

Every candidate carries a priority, the base value of its type in Table 4 plus a bonus (`pii_priority()`). The bonus is 200 for a type forced by the structure, 150 for a user pattern, up to 12 for a national identifier, 10 for a user term, a public body or a learned span of confidence at least 0.7, and 8 for a labeled value. The resolution sorts candidates by priority, length and position and accepts each one that overlaps no accepted candidate. Validated types start at 75 to 100, against 50 to 62 for persons, organizations, places and addresses, so a learned span of these types never displaces a checksum-validated value. By inference from the arithmetic, a learned `USERNAME` of confidence 0.7 reaches 80, the base value of a phone number, so the rule holds strictly for the four learned types only.

Type	Base priority
`EMAIL`	100
`URL`	95
`IBAN`, `CARD`, `NIR`	92
`SIRET`	88
`IPV6`	86
`IPV4`, `MAC`	85
`GEO`	84
`PHONE`	80
`PLATE`	78
`DATE`	75
`GENDER`, `NATIONALITY`	72
`ID`	70
`ADDRESS`	62
`LOCATION`	60
`TITLE`	58
`ORG`	55
`PERSON`	50

Table 4. Base priorities of the fusion, from `pii_priority()` in `dlib/privacy/pii_detector.h` [5].

The recognizers are combined before the fusion (`combine_pii_recognizers()`). The span model leads with its spans of confidence at least 0.5. The tagger adds its spans of at least 0.9 where the span model found nothing and its name parts that complete an accepted name, because its word features also fire on capitalized words that are not personal data (`examples/Readme.md`). Readings between 0.2 and 0.5 serve as weak evidence, so that a capitalized word joined by a conjunction to a person becomes a person with confidence 0.6 (`attach_coordinated_names()`).

Document propagation comes from the first reading (`learn_piece()`). It records the persons found with strong evidence, as full names and as single name words, the places and organizations found with at least 0.8, the acronyms defined in parentheses, the e-mail domains and the software versions. In the second reading, a learned name word turns a bare `Dupont` into a person, learned terms match with confidence 0.85, and learned versions are never read as dates. Filters then drop spans that name nothing. `generic_span()` drops values made only of generic words, such as `Politique de sauvegarde`, unless a given name, a civility, a label or a head noun keeps them, as in `Groupe Danone`, and `spurious_learned_span()` drops learned pieces such as word fragments or file names.

On the synthetic sentence "Our lawyers Martin and Dupont signed the contract with Veolia Water in 2019.", the detector with the current model files reports `Martin` as a person (0.839), `Dupont` through the coordination rule (0.6), `Veolia Water` as an organization (0.914) and `2019` as a date, while the rules alone find only the organization and the date. In "Rose, Violette et Capucine fleurissent au jardin de Mme Claire Martin.", the tagger reads `Capucine` as a given name with 0.903 and the detector reports it as a person. The coverage matrix names such given names used as common nouns as the main residual false positives [9].

3.7 Document handlers

The class `document_anonymizer` applies one detector and one pseudonymizer to text, delimited, markup, Word, Excel, PowerPoint and PDF files, and it refuses binary or password-protected Office files (`dlib/privacy/document_anonymizer_abstract.h`). Every handler runs both readings over the whole document and passes its structure to the detector, so a name established in a labeled cell is replaced in its bare mentions elsewhere. The text handler keeps the encoding, a delimited file is read cell by cell with header rows that type their columns, and in markup only character data, comments and attribute values are rewritten. For these formats, restoration gives back the original bytes when every tag verifies.

The Office handler rebuilds the package with only personal data and pictures changed, drops unreferenced parts and refuses a malformed part (`dlib/privacy/ooxml_anonymizer_abstract.h`). It treats deleted revisions, field codes, comments, notes, headers, text boxes, shared strings, formula results, charts and revision logs, replaces metadata, shifts typed dates in place, renames sheets named after persons, and removes pictures, OLE objects, macros and thumbnails. The PDF handler extracts positioned text and form values and joins paragraph lines, so that a name broken over two lines is found (`dlib/privacy/pdf_anonymizer_abstract.h`). It writes a new PDF that keeps vector graphics, colors and tables and redraws each text run with a standard font, condensed or expanded to its width. Fonts, annotations, attachments, scripts, metadata and earlier revisions are always left out, encrypted

PDFs are refused, and characters outside WinAnsi and Latin Extended-A become `?`.

4. Format-preserving pseudonymization in dlib

4.1 Tags, check codes and substitutes

The class `pseudonymizer` of `dlib/privacy/pseudonymizer.h` replaces each detected value by a tag `[[KEYWORD:BODY#CHK]]` [5]. The keyword names the type and selects the codec, the body is the format-preserving encryption of the value, and the check holds the 15 leading bits of an HMAC-SHA-256 [12, 13] over the keyword and the original value, written as three Crockford base-32 characters (`check_code()`). Figure 5 shows the tag of the synthetic name `Jean Dupont` produced by a demonstration program built against the headers, under the synthetic key 00 01 ... 1f and the context `demo`, which produced every example of Section 4. The header states that holding the key "takes the place of the correspondence table other tools write next to the protected document". The same type and value always give the same tag under one key and context, except civilities, sexes and nationalities.

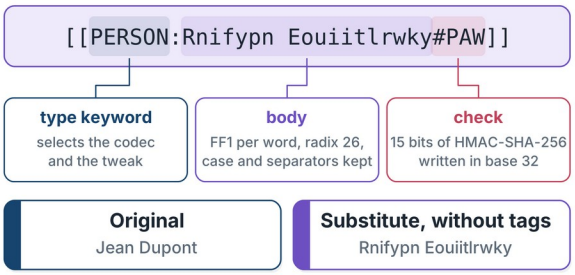


Figure 5. Anatomy of the tag of the synthetic name Jean Dupont: the type keyword selects the codec and its tweak, the body encrypts each word with FF1 in radix 26 while keeping case and separators, and the check holds 15 bits of HMAC-SHA-256 in base 32; without tags, the body alone is written as the substitute.

`decode()` accepts a tag only when its frame, keyword and three check characters are well formed and the recomputed check matches in constant time, and `restore()` leaves a failing candidate in place and counts it as rejected. In the demonstration, one altered letter gives "restored 2, rejected 1", and another key or another context restores none of three tags. By inference from the length of the check, a random tag passes with a probability of about 1 in 32 768 per reading. A substitute is the body written alone, and a shifted date is written in the form of its original (`substitute()`). It carries no check, so the caller keeps its tag and gives a resolver to `find_substitutes()`, which looks up runs of one to twelve words, longest first, and accepts the first tag that verifies. In the demonstration, `Contrat de Jean Dupont, le 12/03/2024, joignable au 06 12 34 56 78.` becomes `Contrat de Rnifypn Eouiitlrwky, le 29/08/2023, joignable au 06 07 09 00 95.`, which the three recorded pairs restore and `restore()` alone leaves unchanged.

4.2 FF1 over AES-256

The class `ff1_cipher` of `dlib/crypto/ff1.h` implements FF1 of NIST SP 800-38G [6], written against the second public draft of its first revision, dated February 2025 [7]. FF1 is a Feistel network of ten rounds whose round function is built on AES in CBC-MAC mode. The code enforces $\text{radix}^n \geq 1\,000\,000$, the domain floor that the draft makes mandatory, and uses exact integer arithmetic, since the draft disallows floating point. Figure 6 shows the rounds. The input splits into A of $u = \lfloor n/2 \rfloor$ numerals and B of $v = n - u$ numerals. Each round encrypts a block that encodes the radix, the lengths, the tweak, the round number and B, turns d bytes of the result into an integer y, replaces A by $(\text{NUM}(A) + y) \bmod \text{radix}^m$, with m alternating between u and v, and swaps the halves. Decryption runs the rounds backwards with a subtraction.

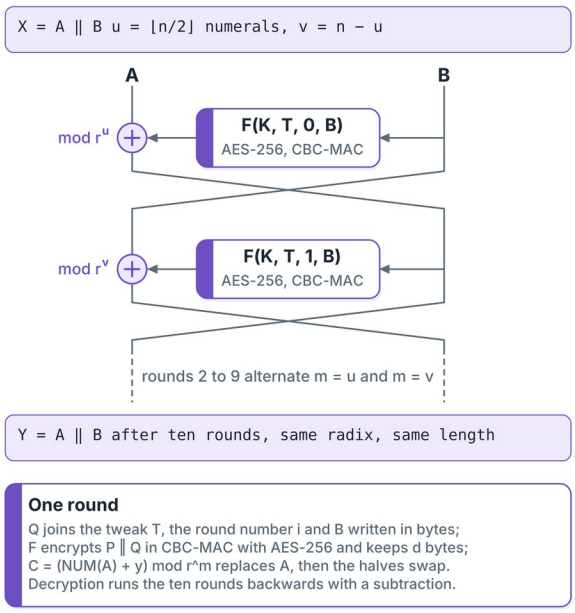


Figure 6. The FF1 Feistel ladder: halves A and B, the round function F built on AES-256 in CBC-MAC mode, the modular addition modulo radix^u or radix^v, and ten rounds that keep the radix and the length.

AES lives in `dlib/crypto/aes.h`, in the encryption direction only [11], with AES-NI when the processor offers it and a portable path without branches or table reads that depend on the data. Every FF1 instance receives a 32-byte sub-key, so FF1 runs on AES-256. The test `test_crypto()` of `dlib/test/privacy.cpp` checks the NIST FF1 samples 1 to 9, AES, SHA-256, HMAC against RFC 4231 [24] and HKDF against RFC 5869 [14], and the demonstration reproduces FF1 sample 7, in which `0123456789` encrypts to `6657667009`. The module implements neither FF3 nor FF3-1, which the 2025 draft removes [7]. The pseudonymizer runs five FF1 instances, in radix 26 for words, 10 for digits and 16, 36 and 62 for hexadecimal and alphanumeric codes, with fixed tweak labels such as `word`, `PHONE` or `CARD`. By inference, fixed tweaks depart from the advice of the draft to vary tweaks [7] and trade that margin for equality of codes, which keeps joins between documents under one key and context at the cost of frequency analysis on values with few forms.

4.3 Key derivation

The pseudonymizer starts from a 256-bit master key, drawn from the random generator of the operating system or derived from a passphrase with PBKDF2-HMAC-SHA-256 over 600 000 iterations [15] (`pseudonymization_key`). In Pseudalis, the engine receives the pseudonymization key derived by the service (Section 2.4) with the context `default` (`engine/engine.cpp`). `derive()` expands it with HKDF-SHA-256 [14], with the salt `dlib.privacy.v1|` followed by the context and the purpose as info, into six sub-keys for words, digits, alphanumeric codes, IP addresses, checks and dates. By inference, another context yields unrelated sub-keys, which is why the demonstration restores nothing under another context. The fingerprint, 12 hexadecimal digits of an HMAC under the check key, identifies the pair of key and context without revealing either, and the demonstration key gives `7de0c73bb1e3`. The date sub-key fixes one secret shift per key and context. The first 8 bytes of an HMAC of `date-shift`, read as an integer v, give a shift of -7 × (1 + v mod 260) days, and months and years move by their nearest whole number, never less than one. The demonstration key gives 196 days, or 28 weeks, 6 months and 1 year into the past.

4.4 Strings and names

Persons, organizations, places, street addresses, e-mail addresses and web addresses share one word codec (`encode_words()`). It keeps separators such as spaces, hyphens, apostrophes and punctuation, lower-cases each word and reads its UTF-8 bytes, padded to at least three, as one integer written in m base-26 numerals. The count m is the smallest with $26^m \geq 256^n$ for n bytes, so the length of the code gives the byte length back. The case of the word, lower, title or upper, is applied to the code, and a word of mixed case is encrypted exactly with a final `!`. Table 5 shows the codec. A person name keeps no word, because particles such as `Le` or `van` would tell the language of the name, while organizations keep legal forms and linking words, places words such as `Saint`, and addresses street types. The tweak carries no type, so `Dupont` gets the code `Eouiitlrwky` in a person, an organization or an e-mail address alike. Codes are always ASCII letters, and their length follows the bytes of the word: `Li` gives 6 letters, `Jean` 7 and `Hélène` 14. By inference, a protected text still tells a short name from a long one.

Original	Substitute
`Jean Dupont`	`Rnifypn Eouiitlrwky`
`DUPONT`	`EOUIITLRWKY`
`M. Dupont`	`Apwcxj. Eouiitlrwky`
`Li`	`Pxxbjr`
`McDonald`	`iupekxevndkaev!`
`Hélène Le Goff`	`Wpazbjtnkkhvru Mqywlv Ykldncw`
`Atelier Dupont et Fils SARL`	`Xkekwmufwmcj Eouiitlrwky et Wldwdst SARL`
`35000 Rennes`	`vnqgrgqtq Gmsiibtmdki`
`Saint-Malo`	`Saint-Ntaahgp`
`12 bis rue des Lilas`	`iaaehw bis rue des Grkhxhkqs`

Table 5. Word codec on synthetic names, organizations, places and addresses, from the demonstration program under a synthetic key [5].

4.5 Numbers and identifiers with check digits

Numbers with check digits keep their layout and receive valid new check digits (Table 6). A card keeps its first six digits and recomputes its Luhn digit, and an IBAN keeps its country code and recomputes its two check digits. A NIR has its 13 digits encrypted and its key recomputed, and a SIRET keeps its establishment number while its SIREN gets the same code as when it stands alone. Alphanumeric identifiers such as a VIN use FF1 in radix 36 or 62 with cycle walking, which repeats the encryption until the result stays in the class of the input. The demonstration checks that the protected card, IBANs, NIRs and SIRET pass their checks, and that the RIB key inside the French BBAN turns invalid, since only the IBAN digits are recomputed. A value that its codec refuses, such as a card that fails Luhn, silently takes the opaque code, FF1 in radix 26 written in lower case with a final `!`. By inference, format preservation is syntactic. The protected NIRs start with 3 and 9 and carry the month fields 14 and 43, so their implausibility shows that a value was pseudonymized, and the parts kept in clear, such as a card issuer or the bank code `NWBK`, can reveal the environment of the holder.

Original	Substitute
card `4970 1012 3456 7893`	`4970 1065 4873 4584`
IBAN `FR76 3000 6000 0112 3456 7890 189`	`FR11 8659 6000 0075 8609 2518 308`
IBAN `GB29 NWBK 6016 1331 9268 19`	`GB45 NWBK 6152 7186 9288 16`
NIR `1 85 03 75 123 456 41`	`3 88 14 42 078 158 84`
NIR `2 69 05 2A 123 456 88`	`9 07 43 2A 973 863 78`
SIRET `123 456 782 00010`	`148 628 936 00012`
IMEI `351234567890124`	`889010107328127`
file number `2024-00123`	`9865-95692`
VIN `VF7AB1234CD567890`	`3N7L3OB9554CIHGIO`
card failing Luhn `4970 1012 3456 7899`	`uzsopuledhshjtlucctjeutjxjjanoqcycw!`

Table 6. Codecs of numbers and identifiers on synthetic values, from the demonstration program under a synthetic key [5].

4.6 E-mail addresses and phone numbers

E-mail addresses go through the word codec with their separators and top-level domain kept, so their codes match those of the names they contain (Table 7). Phone numbers go through FF1 in radix 10 (`code_phone()`). The codec keeps every character that is not a digit, the `+` with the country code and the first national digit, or the trunk `0` with one digit, and it always encrypts at least six digits. In `+33 (0)6 12 34 56 78`, the `(0)` counts as the first national digit, so the mobile digit `6` is encrypted, while it stays readable in `06 12 34 56 78`.

Original	Substitute
jean.dupont@example.com	rnifypn.eouiitlrwky@pspbshklkspj.com
J.Dupont+news@Mail.Example.COM	Fmgdlb.Eouiitlrwky+sxdunuf@Hbqluoy.Pspbshklkspj.COM
06 12 34 56 78	06 07 09 00 95
+33 6 12 34 56 78	+33 6 07 09 00 95
+33 (0)6 12 34 56 78	+33 (0)4 67 27 95 60

Table 7. E-mail addresses and phone numbers, from the demonstration program under a synthetic key [5].

4.7 Dates

Under the default policy, every full date moves into the past by the same secret number of weeks, so order, intervals, ages and weekdays survive. A month with its year and a year alone move by whole months or years, so that no date without its day gives the shift away (`dlib/privacy/pseudonymizer_abstract.h`). The body holds the ISO date and a code of the original layout, as in `[[DATE:2018-08-28~0050#GQN]]` for `12/03/2019`, and the substitute is written back in the layout, language and case of the original (Table 8). A date that does not render back to the same text takes the opaque code. The demonstration keeps the interval from `2026-10-14` to `2026-10-21` at 7 days. The coverage matrix states that a month alone, a season, a duration, a relative date and a year without a cue stay readable, and that a shift of 1 to 260 weeks can leave a future appointment in the future [9]. By inference, one known pair of an original and a shifted date reveals the shift of every date under the same key and context.

Original	Substitute
12/03/2019	28/08/2018
3 mars 1985	19 août 1984
March 1st, 1985	August 17th, 1984
12. März 2024	29. August 2023
2026-10-14	2026-04-01
mars 2022	septembre 2021
2019	2018

Table 8. Dates under the default policy with a shift of 28 weeks, 6 months and 1 year, from the demonstration program [5].

4.8 IP and MAC addresses

IP addresses use Crypto-PAn [25], a prefix-preserving scheme in which two addresses that share a prefix of k bits keep a shared prefix of k bits (`dlib/crypto/prefix_preserving_ip.h`). The code keys AES-128 with one half of the `ip` sub-key and derives its pad from the other half, as the reference implementation does, and an IPv6 address must equal its canonical text form. A MAC address keeps its vendor prefix and encrypts its six device digits in radix 16, with cycle walking that keeps the class of the input (`code_mac()`). In Table 9, the first two IPv4 addresses share 30 leading bits before and after encryption, and the first and third share 16. The tests check only properties of the scheme, so agreement with the reference implementation remains unverified. The reference encrypts 32-bit addresses only, so by inference the IPv6 codes cannot be compared with those of other tools.

Original	Substitute
192.168.12.45	2.176.115.162
192.168.12.46	2.176.115.161
192.168.200.7	2.176.201.183
2001:db8::1	a7fe:8db8:7f6f:f:7fe2:fc7f:fbc8:fefd
00:1A:2B:3C:4D:5E	00:1A:2B:38:8D:86

Table 9. IP and MAC addresses, from the demonstration program under a synthetic key [5].

4.9 The rule applied to each kind of data

Table 10 summarizes the rule applied to each kind of data. GPS coordinates keep their signs and hemispheres and have every digit encrypted, without range checks. A plate such as `AB-123-CD` becomes `EV-390-ZQ` through cycle walking, and a UUID keeps its hyphens and case. Civilities, sexes and nationalities receive a random 20-bit nonce mixed into the tweak and the check, and the value is padded to a multiple of 16 bytes, so `M.` and `Madame` both give bodies of 32 characters and two tags of `Madame` differ.

Kind of data	Rule	Kept readable
names, organizations, places, addresses	FF1 radix 26 per word	case, separators, word lengths, legal forms, street types
e-mail, web address	word codec	separators, top-level domain
phone	FF1 radix 10	country code, first national digit
card	FF1 radix 10, Luhn recomputed	first 6 digits
IBAN	FF1 radix 10 and 26, check recomputed	country, short bank codes
NIR	FF1 radix 10, key recomputed	Corsican letters
SIREN, SIRET	FF1 radix 10, Luhn recomputed	NIC
identifier	FF1 radix 10, 36 or 62	separators, length
UUID, MAC	FF1 radix 16	hyphens, vendor prefix
plate, GPS	FF1 radix 10	layout, signs
IPv4, IPv6	Crypto-PAn	shared prefixes
date	secret shift	order, intervals, weekdays
civility, sex, nationality	FF1 with random nonce	length bucket
value refused by its codec	opaque FF1 radix 26	equality

Table 10. Rule applied to each kind of data by `dlib/privacy/pseudonymizer.h` [5].

5. Synthetic dataset

5.1 The generator and its resources

The generator starts from the premise that annotated personal data cannot be collected without collecting personal data, which

the recognizers exist to avoid. It is the Python program `examples/slm_tools/build_pii_dataset.py` [26], under the Boost Software License 1.0, and every value it writes "is drawn from lexicons … or made up by the procedural generators", so no line describes a real person. The header `dlib/privacy/pii_dataset.h` holds the line format, the BIO mapping, the coarsening of labels and the scorer [5]. The resources stay outside the repository, as 19 files per language of names, places, organizations, templates, context sentences and negatives. Figure 7 shows the chain from these resources to the engine.

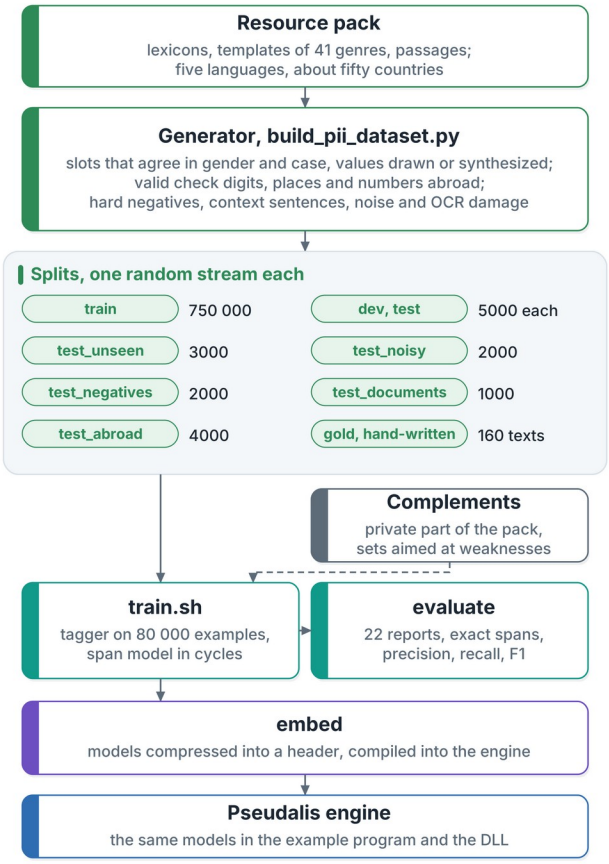


Figure 7. From the resource pack to the engine: the generator, its splits with their sizes, the complementary sets kept outside the repository, training by `train.sh`, evaluation, and the embedding of the models into the engine.

Templates of 41 genres, from forms, letters and e-mails to medical, banking, legal and log texts, hold slots whose identifiers keep one entity across a text, with gender agreement, elision and German cases. Given names come from about thirty traditions, and a quarter of the family names are synthesized from the morphology of about twenty traditions, "so that a model learns what a name looks like rather than a list". Every number carries a valid check, whether an IBAN, a Luhn number, a NIR key or the check digit of one of 31 countries. Hard negatives place capitalized words that are not personal data, such as laws, holidays, regions and common nouns that look like given names, in every genre. Context sentences surround about 30% of the examples, and noise styles add capitals, stripped accents or OCR damage. Texts are assembled from labeled fragments, so spans follow every transformation, and `check_example()` asserts their validity.

5.2 Splits, annotation and hand-written texts

The build script generates the five languages with a seed of 1 and the split sizes of Table 11, each split with its own random stream, and it replaces any evaluation text that repeats a training text. A hash marks 15% of the lexicon entries as held out for `test_unseen`. Spans use byte offsets of the UTF-8 text, never overlap and never cross a tabulation or a line break. The annotation is complete over 24 labels, the 7 that the models learn and 17 kept for scoring. Civilities stay outside names, every mention of a city is labeled, and job titles, regions, countries, products, laws, amounts and ages never are. The 160 gold texts are hand-written, 32 per language, with 2299 spans, and serve evaluation only. Further complementary sets exist outside the repository, and the training set of the current span model joins them to the generated split for 1 207 499 examples [10].

Split	Examples	Spans	Content
`train`	750 000	4 623 319	seen values
`dev`	5000	32 023	seen values
`test`	5000	32 118	all values
`test_unseen`	3000	19 173	held-out values
`test_noisy`	2000	12 619	six times the noise
`test_negatives`	2000	853	hard negatives
`test_documents`	1000	29 877	3 to 6 templates per text
`test_abroad`	4000	25 180	places abroad only
gold texts	160	2299	hand-written

Table 11. Splits of the synthetic dataset and the hand-written texts, with their sizes as counted in the files [10, 26].

6. Performance

6.1 Evaluation protocol

The command `evaluate` of the example program scores, in one run, the rules alone, each recognizer alone and the whole detector with its default options (`examples/slm_document_anonymizer_ex.cpp`). It coarsens references to the replacement level, mapping names to `PERSON`, postal codes and cities to `LOCATION`, street addresses to `ADDRESS` and organizations to `ORG`. A prediction counts only with the same label and the same first and last bytes, scores are micro-averaged, and the time covers the loop over the examples, without model loading. F1 below means the whole detector, every category and exact spans.

6.2 Results of the current models

On `test`, the whole detector with the current models finds 24 622 values, with 1168 false positives and 1699 misses, for a precision of 0.955, a recall of 0.935 and an F1 of 0.945 [10]. Figure 8 compares each split with the first models, trained on the French and English corpus. On `test.fr` and `test.en` the two points overlap, since the models differ there by less than 0.002.

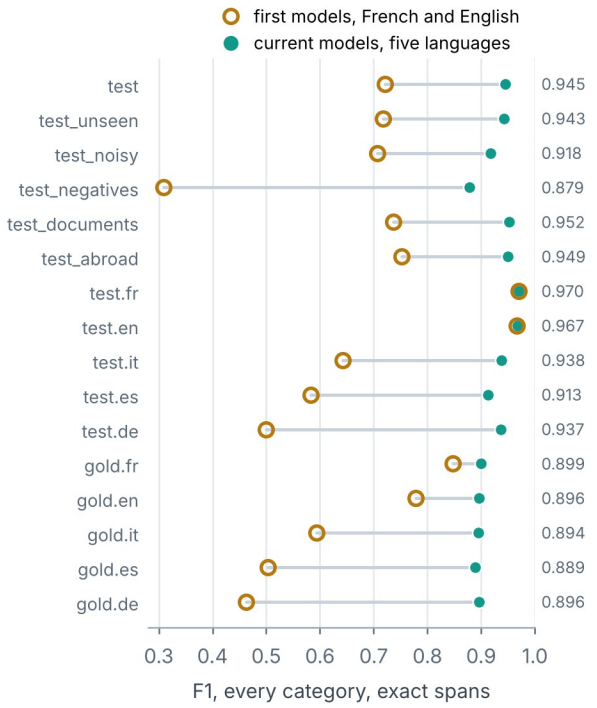


Figure 8. F1 per split, every category and exact spans, for the first models, trained on the French and English corpus, and for the current five-language models.

Table 12 gives the public splits. The languages range from 0.913 for Spanish to 0.970 for French, the hand-written texts from 0.889 to 0.899, and noise costs recall, which falls to 0.896 on `test_noisy`. The split `test_negatives` holds 853 reference spans, and its 201 false positives comprise 89 persons, 56 organizations, 29 places, 4 addresses and 23 dates.

Split	Precision	Recall	F1
`test`	0.9547	0.9355	0.9450
`test_unseen`	0.9513	0.9348	0.9430
`test_noisy`	0.9407	0.8965	0.9181
`test_negatives`	0.8043	0.9683	0.8787
`test_documents`	0.9614	0.9431	0.9522
`test_abroad`	0.9601	0.9390	0.9494
`test.fr`	0.9746	0.9648	0.9697
`test.en`	0.9713	0.9637	0.9675
`test.it`	0.9504	0.9259	0.9380
`test.es`	0.9280	0.8975	0.9125
`test.de`	0.9486	0.9258	0.9371
`gold.fr`	0.9201	0.8793	0.8992
`gold.en`	0.8980	0.8934	0.8957
`gold.it`	0.8994	0.8895	0.8944
`gold.es`	0.8939	0.8840	0.8889
`gold.de`	0.8869	0.9051	0.8959

Table 12. Whole detector with the current models, every category and exact spans, on the public splits [10].

Figure 9 gives the F1 of each category on `test`. Nine categories reach 1.000, while organizations (0.894), identifiers (0.896) and nationalities (0.904) close the ranking. Persons, organizations, places and addresses account for 1019 of the 1168 false positives and 1105 of the 1699 misses. On the hand-written texts, organizations range from 0.683 in German to 0.844 in Spanish, on 39 to 46 values per language.

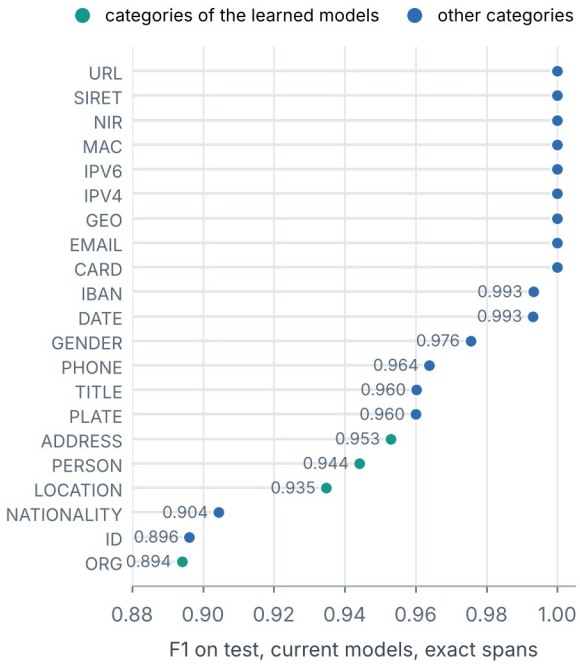


Figure 9. F1 per category on test with the current models, exact spans, for the categories of the learned models and the other categories.

6.3 Components alone

Table 13 isolates the components on `test`. The rules alone reach 0.840, with a precision of 0.947 and a recall of 0.755, so the learned recognizers mainly add recall. The span model alone reaches 0.903 on the fine labels, against 0.749 for the tagger. In one run of the report, the rules took 1.50 s, the tagger 1.55 s, the span model 19.65 s and the whole detector 23.19 s, so the span model takes about 85% of the time.

System	Labels	F1
rules alone	every category	0.8398
rules alone	learned categories	0.7632
tagger alone	fine labels	0.7485
span model alone	fine labels	0.9034
span model alone	learned categories	0.9200
whole detector	learned categories	0.9351
whole detector	every category	0.9450

Table 13. Components alone and combined on `test` with the current models, exact spans [10].

6.4 History from the first models to the current ones

The first models reached 0.721 on `test`, 0.499 on `test.de` and 0.309 on `test_negatives`, with 2797 false positives on that split [10]. The release that introduced the five-language corpus retrained both models with five-language rules and tables, which raised `test` to 0.943 and `gold.de` from 0.462 to

0.903 and cut those false positives to 256. Table 14 follows the later releases, which added rules for coordinated names, the imposed terms and rules for documents of services and technical documents [9]. The false positives of `test_negatives` went on to 232, 218 and 201. The current release changed no rule and retrained both models. Against the previous models, F1 rose on 15 of the 22 evaluated splits and false positives fell on 18 of them. The largest losses are on `test_abroad`, where persons of other countries lost 29 found values, and on `gold.es`, with three values lost and six false positives gained [10].

Split	Five-language release	Previous models	Current models
`test`	0.9426	0.9440	0.9450
`test_unseen`	0.9419	0.9431	0.9430
`test_noisy`	0.9158	0.9173	0.9181
`test_negatives`	0.8549	0.8708	0.8787
`test_documents`	0.9508	0.9515	0.9522
`test_abroad`	0.9477	0.9509	0.9494
`test.fr`	0.9659	0.9684	0.9697
`test.en`	0.9674	0.9678	0.9675
`test.it`	0.9365	0.9374	0.9380
`test.es`	0.9097	0.9108	0.9125
`test.de`	0.9335	0.9357	0.9371
`gold.fr`	0.8837	0.8942	0.8992
`gold.en`	0.8844	0.8980	0.8957
`gold.it`	0.8990	0.8907	0.8944
`gold.es`	0.8849	0.9010	0.8889
`gold.de`	0.9029	0.8962	0.8959

Table 14. F1 of the whole detector, every category and exact spans, for the release that introduced the five-language corpus, the previous models and the current models [10].

6.5 Real documents and complementary sets

On a private sample of real documents, the current models reach a precision of 0.872 and a recall of 0.917, against 0.875 and 0.910 for the previous models [9]. On the complementary synthetic test sets kept outside the repository, their F1 ranges from 0.817 to 0.930 [10]. By inference from the recall on real documents, about one personal value in twelve stays in clear in such documents, which supports the reading before sharing that the README and the coverage matrix recommend [1, 9].

6.6 Speed

On 8 November 2026, on a virtual machine with two cores, the whole detector read the 961 830 characters of `test`, the synthetic test split, in 23.4 to 23.7 s with the current models, against 24.4 to 24.5 s with the previous models, and the rules alone took 1.50 to 1.68 s. That is about 41 000 characters per second for the whole detector, with 2.9% to 4.5% less time than the previous models. Only runs paired in one session compare releases, since one program took 55.1 to 55.9 s in one session and 26.0 to 27.8 s in another (Table 15) [10].

Pair of releases	Newer	Older
current and previous models	23.4 to 23.7 s	24.4 to 24.5 s
previous models and their predecessor	26.5 to 28.9 s	26.0 to 27.8 s
intermediate pair, idle machine	55.1 to 55.9 s	54.6 to 55.2 s
intermediate pair, shared core	27.4 s	30.2 s
five-language release and first models	33.0 to 34.2 s	28.0 to 32.3 s

Table 15. Time of the whole detector on `test` for releases measured one after the other in the same session; rows are not comparable with each other [10].

The service itself, on its Linux build, processed synthetic documents in three runs each (Table 16). A text of 5000 words in which it replaced 720 values took 2.48 to 2.62 s as a `.txt` file and 1.37 to 1.46 s as a `.docx` file holding the same text, and a CSV file of 1000 rows with 5027 values took 5.75 to 5.88 s. These times cover the whole request to the service. The runs do not explain why the Word file goes faster than the same text in a plain file.

Document	Size	Time
text, 5000 words, `.txt`	27.5 KB	2.48 to 2.62 s
same text, `.docx`	27.5 KB of text	1.37 to 1.46 s
CSV, 1000 rows	95 KB	5.75 to 5.88 s

Table 16. End-to-end time of the service on synthetic documents, Linux build, three runs each, measured on 8 November 2026.

7. GDPR coverage

7.1 Definitions and the status of the output

Support for compliance with the GDPR is a stated goal of Pseudalis [1, 9]. The mapping below takes the legal texts as they stood on 8 November 2026, as a technical description that is not legal advice. The qualification of a processing, its legal basis and the need for an impact assessment remain with the controller, under supervisory authorities and courts.

Article 4(1) of the Regulation defines personal data as "any information relating to an identified or identifiable natural person" [8]. Article 4(5) defines pseudonymisation as processing after which the data "can no longer be attributed to a specific data subject without the use of additional information", provided that this information is kept separately and protected. Recital 26 states that such data "should be considered to be information on an identifiable natural person", and that the principles of data protection do not apply to anonymous information.

The README qualifies the output as a pseudonymization in the sense of the GDPR, since the key restores the data, which stay personal data for its holder [1]. The additional information is the master key with the vault and the database of substitutes. An inference consistent with the design concerns output without tags. A substitute is the FF1 body of its tag, deterministic for a key and a context, so whoever holds the key can decrypt it or

confirm a guess without the record, and a file protected without tags and without a record remains pseudonymized data for the key holder. The example program offers an irreversible mode that protects with a random key it never stores (`examples/slm_document_anonymizer_ex.cpp`), and whether its output is anonymous then depends on what the remaining text reveals. Recital 14 leaves legal persons outside the Regulation, and Pseudalis still replaces organizations by default. The coverage matrix explains that the name of an organization in a document about a person designates the employer, the fund, the court or the agency of that person and often suffices to re-identify the person [9].

7.2 Categories covered, partly covered and not covered

Table 17 condenses the coverage matrix [9]. Direct identifiers, contact data, identifiers with a verifiable check, online and vehicle identifiers and organizations are covered. Identifiers recognizable only through a label are partly covered. Categories stated in whole sentences, quasi-identifiers and amounts stay readable. The matrix explains that the special categories of Article 9 and the criminal data of Article 10 appear in sentences such as "suivi pour un diabète de type 2", which a tag cannot replace without destroying their meaning, and that a sentence classifier to flag them for review "remains to be built". By default, organizations, public bodies, civilities, sexes and nationalities are replaced.

Category	State	Means
names, nicknames, logins	covered	rules, labels, propagation, models
addresses, cities, postal codes with city	covered	rules, labels, models
phone, e-mail, dates	covered	scanners, dates shifted
civility, sex, nationality	covered	rules, labels
identifiers with a check (NIR, IBAN, cards, about 40 countries)	covered	validated scanners
IP, MAC, UUID, IMEI, plates, VIN, GPS	covered	validated scanners
organizations, named public bodies	covered	rules, tables, models, imposed terms
identity card, driving license, residence permit	partly	label only
customer, case, contract, employee numbers	partly	label, cue or pattern
patient numbers, foreign plates	partly	label or cue
postal code alone, region, country	not covered	left readable
age, amounts linked to a person	not covered	left readable
health, beliefs, criminal facts	not covered	left readable
session tokens, API keys, device fingerprints	not covered	user patterns only
text inside pictures	not covered	no OCR, picture removed

Table 17. Coverage of categories of personal data by Pseudalis, condensed from the coverage matrix [9].

7.3 Articles 25 and 32

Article 25(1) requires appropriate measures "such as pseudonymisation" at the time of the determination of the means and at the time of the processing, and Article 25(2) limits by default the data processed, the period of their storage and their accessibility [8]. Article 32(1) lists "the pseudonymisation and encryption of personal data", the confidentiality and integrity of systems, the restoration of availability and the regular testing of the measures. Pseudalis provides several of these measures in its code: local processing behind a loopback interface with checks of host and origin, FF1 tags under AES-256, an authenticated encrypted vault with time and space limits and a forget function, a key protected by DPAPI and exported only under a passphrase, imposed terms, signed updates, and the command `evaluate` for testing on an organization's own samples.

Three inferences complete the mapping. The key export is the backup, so a lost key is a loss of availability under Article 32(1)(c). The time limit starts only when a copy leaves the drive, so originals that arrived through the interface or the Explorer commands appear bounded by the space budget and manual forgetting, which bears on the storage limitation of Article 5(1)(e). The trust model fits recital 29, which allows pseudonymization within one controller with authorized persons, only where interactive access to the computer is limited to such persons [1].

7.4 EDPB guidance as of November 2026

The EDPB adopted its Guidelines 01/2025 on Pseudonymisation on 16 January 2025 and published them on 17 January 2025 as version 1 for public consultation [27, 28, 29]. On 8 November 2026 the consultation page still offers version 1 only, with no final version, and in December 2025 the EDPB stated that it was reassessing the work in the light of the case law of the Court [27, 30]. Version 1 holds that pseudonymised data "remains information related to an identifiable natural person and is therefore still personal data" [28]. It describes pseudonymisation in three steps within a pseudonymisation domain and presents it as a measure under Articles 25 and 32 [29]. By inference, the pseudonymisation domain of an installation is everything outside its computer, since every user of the computer lies inside the trust boundary [1].

The Guidelines 02/2026 on Anonymisation remain a draft under consultation, adopted on 7 July 2026 with feedback open from 8 July to 30 October 2026, and no final version was found [31, 32, 33]. The draft tests record isolation, linkage and inference, and when data fail a criterion it requires further analysis, including a contextual approach from the perspective of each relevant entity [32, 33]. By inference, the deterministic codes, prefix-preserving IP codes and shifted dates of one installation bear on linkage, readable health details and amounts bear on record isolation and inference, and the output of the irreversible mode needs a documented assessment before anyone treats it as anonymous.

7.5 Court of Justice cases

In *Breyer* (C-582/14, 2016), the Court held that a dynamic IP address is personal data for a website operator with legal channels to obtain the identity of the subscriber [34, 35], which supports the coverage of IP addresses. In *Scania* (C-319/22, 2023), it held that a vehicle identification number is personal data for whoever may reasonably link it to the holder of the vehicle [36, 37]. In *EDPS v SRB* (C-413/23 P, 2025), it held that pseudonymised data "do not constitute, in all cases and for every person, personal data", while the duty to inform is assessed from the perspective of the controller at collection, so the recipients of pseudonymised data must be named [30, 38, 39]. Commentary reads a decision of the Conseil d'État of 13 February 2026 as applying this test, when it held that pseudonymised health databases holding ages, sexes, pathologies, consultation dates and a unique patient identifier constituted personal data [40]. The CNIL likewise treats pseudonymised data as personal data [41].

Inferences follow for Pseudalis. Every protected file remains personal data for the key holder. A recipient without the key holds personal data unless the readable remainder offers no means reasonably likely to identify a person, which has to be assessed per document and per recipient. The deterministic substitutes of one installation act as persistent pseudonymous identifiers, the configuration that the Conseil d'État held to be personal data when combined with readable health details.

7.6 Limits

The trust boundary is the whole computer, since any interactive user can restore files, read records and export the key, and a file written to the drive lies in clear until it is processed [1]. Detection is probabilistic in the learned categories, with a recall of 0.917 on real documents (Section 6.5), and a reference that no word announces needs a user pattern [9]. Encoded values in texts and values that the detector misses in picture descriptions stay in clear, and the README and the coverage matrix both recommend reading a protected document before sharing it [1, 9]. Codes keep the case, word count and length of their words, and grammatical agreement can reveal the sex of a person whose civility was replaced [9]. By inference, one known date reveals the date shift of a whole installation. An inference that the sources do not discuss concerns check digits. Substitutes of NIRs and IBANs keep valid checks, so they can coincide with real identifiers of other people. Pseudalis is therefore one technical measure within the compliance of a controller, which keeps the legal qualification, the impact assessment and the review before sharing.

8. Code availability

The implementation is part of the public Dlib-Transformer-extensions repository on GitHub, distributed under the Boost Software License 1.0, at https://github.com/Cydral/Dlib-Transformer-extensions [3]. The source code and README of Pseudalis lie at https://github.com/Cydral/Dlib-Transformer-extensions/tree/main/dlib/dlib/external/pseudalis [1], and the installer with its signed feed at https://github.com/Cydral/Dlib-Transformer-extensions/tree/main/dist/pseudalis [2]. The module `dlib/privacy` lies at https://github.com/Cydral/Dlib-Transformer-extensions/tree/main/dlib/dlib/privacy [5], and the dataset generator at https://github.com/Cydral/Dlib-Transformer-extensions/blob/main/examples/slm_tools/build_pii_dataset.py [26]. The trained models are compiled into the engine through the header `examples/slm_document_anonymizer_ex.h`, which the command `embed` of the example program writes. The dataset resources and the complementary sets stay outside the repository. The demonstration program of Section 4 includes only `dlib/privacy/pseudonymizer.h`, builds with `g++ -std=c++14` without any other source file, and is not part of the repository.

9. Conclusion

Pseudalis Community keeps every secret of reversibility on one computer: the master key, the vault of originals and the records of substitutes. Protection with tags keeps restorability inside the document, under the key. Protection without tags moves it into the record, so for such files the retention settings become a parameter of reversibility, and a forgotten record ends it with HTTP 410. An administrator who chooses tagless output for external sharing therefore chooses at the same time how long its originals remain recoverable.

The measurements locate the remaining work in the learned categories, organizations first, since the validated scanners hold the structured categories at or near F1 1.000. The span model of 116 791 parameters carries most of the recall in these categories and about 85% of the detection time, and the evaluation README has named the capacity of this tiny model as the limit that remains since the five-language corpus came in [10]. By inference, gains on organizations and on real documents, where recall stands at 0.917, call for a larger model or for more evidence drawn from the whole document. The FF1 codecs keep syntax, check digits and equality, which keeps protected documents usable and joinable for their recipients. The same equality keeps them linkable, so the status of a protected document for a recipient without the key depends on what its text leaves readable, case by case, as the EDPB draft on anonymisation and the case law of the Court assess it.

References

[1] A. Pierrain. Pseudalis Community, source code and README. Dlib-Transformer-extensions repository on GitHub, https://github.com/Cydral/Dlib-Transformer-extensions/tree/main/dlib/dlib/external/pseudalis, 2026.

[2] A. Pierrain. Pseudalis Community, installer and signed update feed. Dlib-Transformer-extensions repository on GitHub, https://github.com/Cydral/Dlib-Transformer-extensions/tree/main/dist/pseudalis, 2026.

[3] Dlib-Transformer-extensions project. Dlib-Transformer-extensions, transformer extensions of the dlib C++ library. GitHub repository, https://github.com/Cydral/Dlib-Transformer-extensions, 2026.

[4] D. E. King. Dlib-ml: a machine learning toolkit. Journal of Machine Learning Research, vol. 10, pp. 1755-1758, 2009.

[5] Dlib-Transformer-extensions project. The dlib/privacy module for personal-data detection and pseudonymization. Dlib-Transformer-extensions repository on GitHub, https://github.com/Cydral/Dlib-Transformer-extensions/tree/main/dlib/dlib/privacy, 2026.

[6] M. Dworkin. Recommendation for block cipher modes of operation: methods for format-preserving encryption. NIST Special Publication 800-38G, DOI 10.6028/NIST.SP.800-38G, 2016.

[7] M. Dworkin and N. Mouha. Recommendation for block cipher modes of operation: methods for format-preserving encryption, second public draft. NIST Special Publication 800-38G Rev. 1 (draft), DOI 10.6028/NIST.SP.800-38Gr1.2pd, 2025.

[8] European Parliament and Council of the European Union. Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation). Official Journal of the European Union, L 119, pp. 1-88, CELEX 32016R0679, 2016.

[9] Pseudalis project. Personal data, strict definition and coverage of dlib/privacy (coverage matrix, in French), version of 8 November 2026. Internal document, 2026.

[10] Pseudalis project. Evaluation and training reports of the Pseudalis detectors, from the first models to the current ones, with the README of the dataset pack. Dataset pack kept outside the repository, 2026.

[11] National Institute of Standards and Technology. Advanced Encryption Standard (AES). FIPS 197, updated 2023, DOI 10.6028/NIST.FIPS.197-upd1, 2001.

[12] National Institute of Standards and Technology. The keyed-hash message authentication code (HMAC). FIPS 198-1, DOI 10.6028/NIST.FIPS.198-1, 2008.

[13] National Institute of Standards and Technology. Secure Hash Standard (SHS). FIPS 180-4, DOI 10.6028/NIST.FIPS.180-4, 2015.

[14] H. Krawczyk and P. Eronen. HMAC-based extract-and-expand key derivation function (HKDF). IETF RFC 5869, 2010.

[15] OWASP Foundation. Password storage cheat sheet. OWASP Cheat Sheet Series, consulted on 8 November 2026, 2026.

[16] I. Tsochantaridis, T. Joachims, T. Hofmann and Y. Altun. Large margin methods for structured and interdependent output variables. Journal of Machine Learning Research, vol. 6, pp. 1453-1484, 2005.

[17] L. A. Ramshaw and M. P. Marcus. Text chunking using transformation-based learning. Third Workshop on Very Large Corpora, arXiv:cmp-lg/9505040, 1995.

[18] A. Vaswani, N. Shazeer, N. Parmar et al. Attention is all you need. Advances in Neural Information Processing Systems 30, arXiv:1706.03762, 2017.

[19] B. Zhang and R. Sennrich. Root mean square layer normalization. Advances in Neural Information Processing Systems 32, arXiv:1910.07467, 2019.

[20] J. Ainslie, J. Lee-Thorp, M. de Jong et al. GQA: training generalized multi-query transformer models from multi-head checkpoints. Proceedings of EMNLP 2023, arXiv:2305.13245, 2023.

[21] J. Su, M. Ahmed, Y. Lu et al. RoFormer: enhanced transformer with rotary position embedding. Neurocomputing, vol. 568, 127063, 2024.

[22] N. Shazeer. GLU variants improve transformer. arXiv:2002.05202, 2020.

[23] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. International Conference on Learning Representations, arXiv:1711.05101, 2019.

[24] M. Nystrom. Identifiers and test vectors for HMAC-SHA-224, HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512. IETF RFC 4231, 2005.

[25] J. Xu, J. Fan, M. H. Ammar and S. B. Moon. Prefix-preserving IP address anonymization: measurement-based security evaluation and a new cryptography-based scheme. 10th IEEE International Conference on Network Protocols (ICNP), Paris, 2002.

[26] Dlib-Transformer-extensions project. Synthetic dataset generator build_pii_dataset.py. Dlib-Transformer-extensions repository on GitHub, https://github.com/Cydral/Dlib-Transformer-extensions/blob/main/examples/slm_tools/build_pii_dataset.py, 2026.

[27] European Data Protection Board. Guidelines 01/2025 on Pseudonymisation, version 1 for public consultation. EDPB, 2025.

[28] European Data Protection Board. EDPB adopts pseudonymisation guidelines and paves the way to improve cooperation with competition authorities. EDPB news, 17 January 2025, 2025.

[29] Hunton Andrews Kurth LLP. EDPB advises on pseudonymisation for GDPR compliance. Privacy Laws and Business, 5 June 2025, 2025.

[30] European Data Protection Board. Discussion paper for the stakeholder event of 12 December 2025 on anonymisation and pseudonymisation. EDPB, December 2025, 2025.

[31] European Data Protection Board. Guidelines 02/2026 on Anonymisation, version 1 for public consultation. EDPB, 2026.

[32] D. Cooper, K. Van Quathem and A. Bertrand. EDPB publishes draft guidelines on anonymisation. Covington and Burling, Inside Privacy, 4 August 2026, 2026.

[33] Sidley Austin LLP. EDPB publishes draft guidelines on anonymisation. Data Matters, 27 July 2026, 2026.

[34] Court of Justice of the European Union. Judgment of 19 October 2016, Patrick Breyer v Bundesrepublik Deutschland. Case C-582/14, 2016.

[35] Osborne Clarke. European Court rules website and app operators must treat dynamic IP addresses as personal data. Osborne Clarke Insights, 19 October 2016, 2016.

[36] Court of Justice of the European Union. Judgment of 9 November 2023, Gesamtverband Autoteile-Handel e.V. v Scania CV AB. Case C-319/22, 2023.

[37] datenrecht.ch. ECJ in the case of Scania (Case C-319/22), relative approach to determining identifiability. datenrecht.ch, n.d.

[38] Court of Justice of the European Union. Judgment of 4 September 2025, European Data Protection Supervisor v Single Resolution Board. Case C-413/23 P, Official Journal of the European Union C/2025/5551, 2025.

[39] A&O Shearman. CJEU clarifies concept of personal data for a transfer of pseudonymised data to third parties. A&O Shearman on Data, 2025.

[40] Herbert Smith Freehills Kramer. French Supreme Court rules on pseudonymisation vs anonymisation. HSF Kramer data notes, 2026.

[41] Commission nationale de l'informatique et des libertés. Identifier les données personnelles. CNIL, 27 January 2020, 2020.